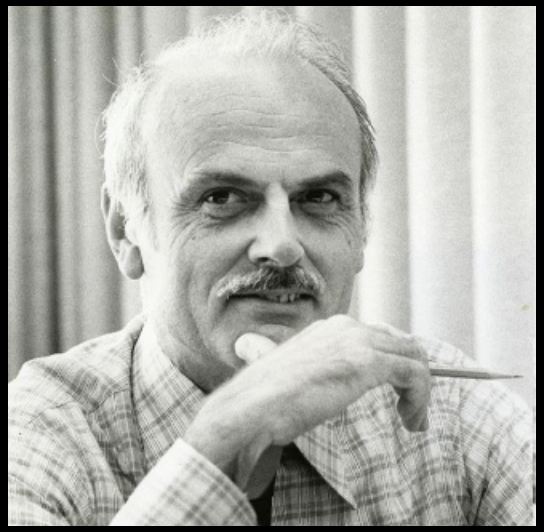
UNIT-3



Relational Data Models

❖ History of Relational Data Model

- ★ First introduced by Ted Codd (in 1970).
- ★ Uses concept of mathematical relation.
- ★ First commercial implementations of the relational model → Oracle DBMS, SQL/DS system (IBM).
- ★ Current popular RDBMSs → SQL Server & Access (Microsoft), DB2 & Informix (IBM), etc.
- ★ Standard for commercial RDBMS → SQL query language.

❖ Terminologies

★ Relational model represents data as a collection of tables.

★ A table is also called a relation.

★ Each row → tuple.

★ Column headers → attributes.

Relation name

Tuples

STUDENT	ROLL_NO	NAME	AGE	PHONE
	1	David	18	2525364236
	2	Benjamin	19	3526699663

❖ Terminologies

★ Domain:

- A set of atomic values allowed for an attribute.
- Ex: (a) Name : string of characters that represent name of persons.

  (b) Employee_ages: Possible ages of employees of a company (values between 20 & 70 years old).

Ensure that every attribute value is atomic (single-valued) and falls within a predefined domain or data type (e.g., an age must be an integer).

❖ Terminologies

★ Relation schema:

- Describes a relation.
- Made up of a relation name R and a list of attributes $A_1, A_2, A_3, \ldots, A_n$.

STUDENT (Name, RollNo, Age, Address, Phone, Grade)

STUDENT (Name: string, RollNo: integer, Age: integer, Address: string, Phone: string, Grade: real)

★ Degree (or arity) of a relation:

Degree = 6

- Number of attributes in a relation schema.

STUDENT (Name, RollNo, Age, Address, Phone, Grade)

STUDENT (Name: string, RollNo: integer, Age: integer, Address: string, Phone: string, Grade: real)

★ Cardinality:

- Total number of tuples preser[redacted] in a relation.

STUDENT	ROLL_NO	NAME	AGE
	1	Harry	19
	2	Ben	22
	3	Kathy	20

Cardinality = 3

★ Relational database schema:

- Is a set of relation schemas and a set of integrity constraints.

★ Relation state (or relation instance):

- Set of tuples at a given time.

STUDENT	ROLL_NO	NAME	AGE
	1	Harry	19
	2	Ben	22
	3	Kathy	20

Cardinality = 3

❖ Characteristics of Relations

★ Ordering of Tuples within a Relation

- A relation is a set of tuples.

Set ⟶ $A = \{1, 2, 3, 4\}$

$B = \{4, 2, 1, 3\}$

- Tuples in a relation need not have any particular order.

STUDENT	ROLL_NO	NAME	AGE
	1	Harry	19
	2	Ben	22
	3	Kathy	20

STUDENT	ROLL_NO	NAME	AGE
	2	Ben	22
	1	Harry	19
	3	Kathy	20

❖ Characteristics of Relations

★ Ordering of Values within a Tuple:

- An n-tuple → ordered list of n values, so ordering of values in a tuple is important.
- With an alternative definition of relation, ordering of values in a tuple is unnecessary.

STUDENT	ROLL_NO	NAME	AGE
	1	Harry	19
	2	Ben	22
	3	Kathy	20

❖ Characteristics of Relations

★ Ordering of Values within a Tuple:

- A tuple → set of (<attribute>, <value>) pair, then ordering of attributes is not important.

STUDENT	ROLL_NO	NAME	AGE
	1	Harry	19
	2	Ben	22
	3	Kathy	20

t = <(RollNo, 2), (Name, Ben), (Age,22)>

t = <(Name, Ben), (Age,22), (RollNo, 2)>

❖ Characteristics of Relations

★ Values & Nulls in a Tuple:

STUDENT	ROLL_NO	NAME	ADDRESS
	2	Ben	Bengaluru, Karnataka-560051

- Each value in a tuple is an atomic value.

STUDENT	ROLL_NO	NAME	CITY	STATE	PINCODE
	2	Ben	Bengaluru	Karnataka	560051

- Nulls → unknown or not applicable.

STUDENT	ROLL_NO	NAME	AGE	MOBILE
	1	Harry	19	null
	2	Ben	22	6523214523
	3	Kathy	null	

242526778

❖ Characteristics of Relations

★ Interpretation of a Relation:

- The relation schema can be represented as a declaration or assertion.

STUDENT (Roll_No, Name, Age, Mobile)

- Each tuple can be interpreted as a fac

STUDENT	ROLL_NO	NAME	AGE	MOBILE
	1	Harry	19	1203571204
	2	Ben	22	6523214523
	3	Kathy	20	2525364562

❖ Relational Model Constraints

★ Constraints on databases:

- Inherent model-based: Inherent in the data model.
- Schema based: Defined directly in the schemas of the data model.
- Application based: Must be expressed and enforced by the application programs.

❖ Schema-based Constraints

★ Domain Constraints:

- Must be an atomic value.
- Performs data type check.

STUDENT	RollNo	Name	Phone	Age
	1	Jeremy	2563251425	14
	2	Charles	1234568978	14
	3	Weston	3625368914	A

Violates Domain Constraints

★ Key Constraints:

- An attribute that can uniquely identify each tuple in a relation is called a key.

❖ Schema-based Constraints

★ Domain Constraints:

- Must be an atomic value.
- Performs data type check.

STUDENT	RollNo	Name	Phone	Age
	1	Jeremy	2563251425	14
	2	Charles	1234568978	14
	3	Weston	3625368914	A

Violates Domain Constraints

★ Key Constraints:

- An attribute that can uniqu[redacted] identify each tuple in a relation is called a key.

Key

STUDENT	RollNo	Name	Age	Grade
	1	[redacted]eremy	14	A
	2	Charles	14	A
	3	Charles	13	

B

❖ Schema-based Constraints

★ Key Constraints:

- A superkey specifies that no two tuples can have the same value.

STUDENT	RollNo	Name	Age	Email
	1	Jeremy	14	jeremy16@gmail.com
	2	Josh	14	josh25@gmail.com
	3	Charles	15	charly01@gmail.com

- Every relation has at least one superkey → set of all attributes.

SK = { RollNo }, { Email }, { RollNo, Name }, { RollNo, Age }, { RollNo, Email }, { Name, Email }, { Age, Email }, { RollNo, Name, Age, Email }

❖ Schema-based Constraints

★ Key Constraints:

- A key satisfies 2 constraints:

  (i) Two tuples cannot have identical values for all the attributes in the key.

  (ii) It is a minimal superkey.

STUDENT	RollNo	Name	Age	Email
	1	Jeremy	14	jeremy16@gmail.com
	2	Josh	14	josh25@gmail.com
	3	Charles	15	charly01@gmail.com
	3	Alicia	13	alicia22@gmail.com

Not possible

SK = { RollNo }, { Email }, { RollNo, Name }, { RollNo, Age }, { RollNo, Email }, { Name, Email }, { Age, Email }, { RollNo, Name, Age, Email }

❖ Schema-based Constraints

★ Key Constraints

- Candidate Keys:

→ Set of attributes that uniquely identify the tuples in a relation.

STUDENT	RollNo	Name	Age	Email
	1	Jeremy	14	jeremy16@gmail.com
	2	Josh	14	josh25@gmail.com
	3	Charles	15	charly01@gmail.com

❖ Schema-based Constraints

★ Constraints on Null Values:

- Specifies whether null values are permitted or not (NOT NULL).

STUDENT	RollNo	Name	Age	Grade
	1	Jeremy	14	A
	2	Charles	14	A
	3	Charles	13	B

★ Entity Integrity Constraint:

- States that no primary key value can be null.

STUDENT	RollNo	Name	Age	Grade
	1	Jeremy	14	A
	2	Charles	14	A
	null	Charles	13	B

❖ Schema-based Constraints

★ Referential Integrity Constraint:

- Specified between 2 relations.
- States that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.

STUDENT	SID	SNAME	DNO
	1001	Alicia	3
	1002	Katherine	1

DEPARTMENT	DNO	DNAME
	1	CS
	2	ECE
	3	CIVIL

❖ Schema-based Constraints

★ Referential Integrity Constraint:

- Foreign key must satisfy the following:

  (i) same domain.

  (ii) Value of FK in a tuple either occurs as a value of PK i.e.,

  $t_1[FK] = t_2[PK]$ or is null.

Foreign Key

STUDENT	SID	SNAME	DNO
	1001	Alicia	3
	1002	Katherine	1

DEPARTMENT	DNO	DNAME
	1	CS
	2	ECE
	3	CIVIL

Codd's 12 Rules

Dr Edgar F. Codd, after his extensive research on the Relational Model of database systems, came up with twelve rules of his own, which according to him, a database must obey in order to be regarded as a true relational database

Codd's Rules

0. The Foundation Rule
1. Information Rule
2. Guaranteed Access Rule
3. Systematic Treatment of Null Values
4. Active/Dynamic Online Catalog based on the relational model
5. Comprehensive Data Sub-Language Rule
6. View Updating Rule
7. Relational Level Operation Rule
8. Physical Data Independence Rule
9. Logical Data Independence Rule
10. Integrity Independence Rule
11. Distribution Independence Rule
12. Non Subversion Rule

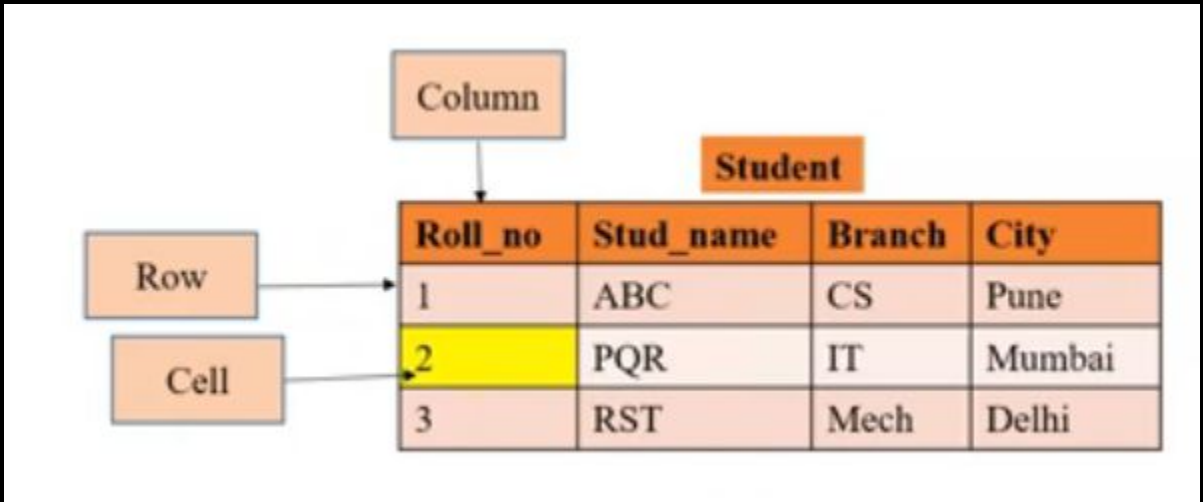
Codd's 12 Rules

- **Rule 0: Foundation rule:**

  For a system to be qualified as a relational database management system, it must be able to manage databases entirely through its relational capabilities.

Codd's 12 Rules

- **Rule 1: Information Rule**:-The data stored in a database, may it be user data or metadata, must be a value of some table cell. Everything in a database must be stored in a table format.



- **Rule 2: Guaranteed Access Rule** :-Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value). No other means, such as pointers, can be used to access data.

- **Rule 3: Systematic Treatment of NULL Values** :-The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following – data is missing, data is not known, or data is not applicable.

Student

Roll_no	Stud_name	Branch	City
1	ABC	CS	Pune
2	PQR	IT	Mumbai
3	RST	Mech	NULL

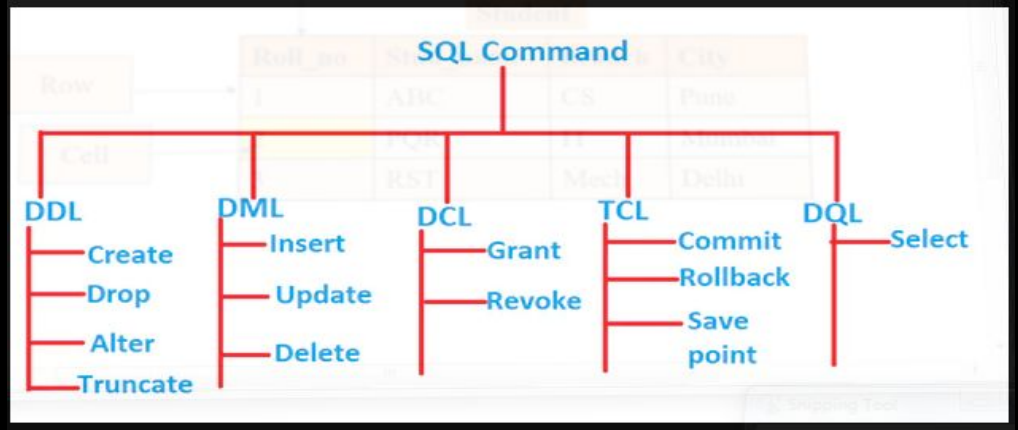
Codd's 12 Rules

- **Rule 4: Active Online Catalog**

  The structure description of the entire database must be stored in an online catalog, known as data dictionary, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself

- **Rule 5: Comprehensive Data Sub-Language Rule**

  A database can only be accessed using a language (like SQL)having linear syntax that supports data definition, data manipulation, and transaction management operations. This language can be used directly or by means of some application. If the database allows access to data without any help of this language, then it is considered as a violation.



Codd's 12 Rules

- **Rule 6: View Updating Rule**

  All the views of a database, which can theoretically be updated, must also be updatable by the system

  Views are the virtual tables created by using queries to show the partial view of the tables.

  That is views are the subset of the table. It is only partial table with few columns and rows.

  This rule states different views created for different purposes should be automatically updated by the system itself.

Codd's 12 Rules

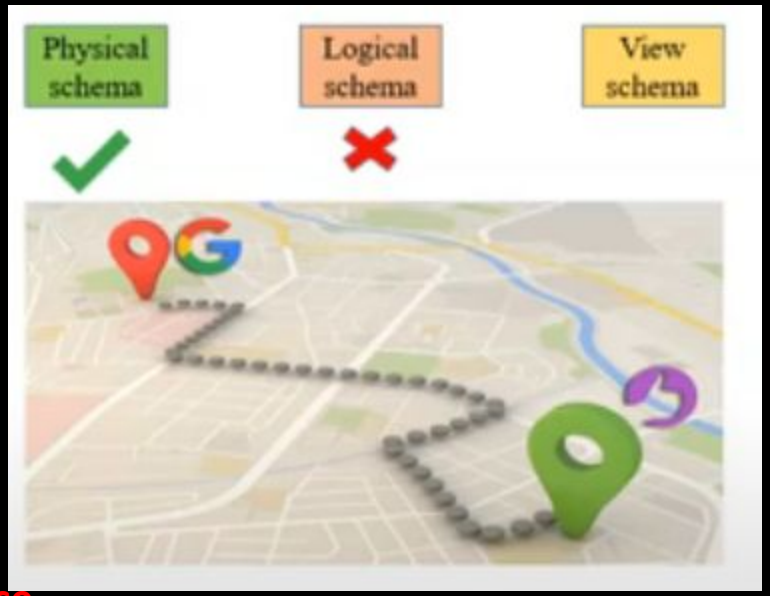
Rule 7: Relational Level Operation

- This rule states that every query language used by the Database should support INSERT. UPDATE, and DELETE on the records. This must not be limited to a single row, that is, it must also support UNION, UNION ALL,INTERSECTION ,INTERSECTION ALL AND MINUS operations to yield sets of data records.

Codd's 12 Rules

Rule 8: Physical Data Independence

- The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.



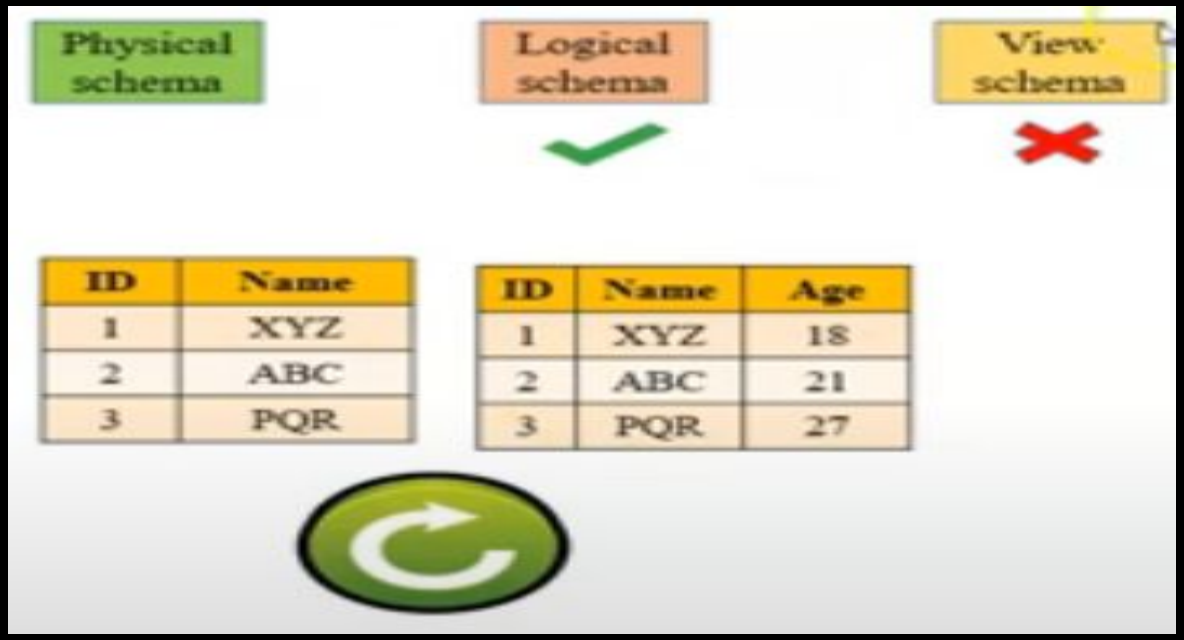
Rule 9: Logical Data Independence

- The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.

Codd's 12 Rules

Rule 9: Logical Data Independence

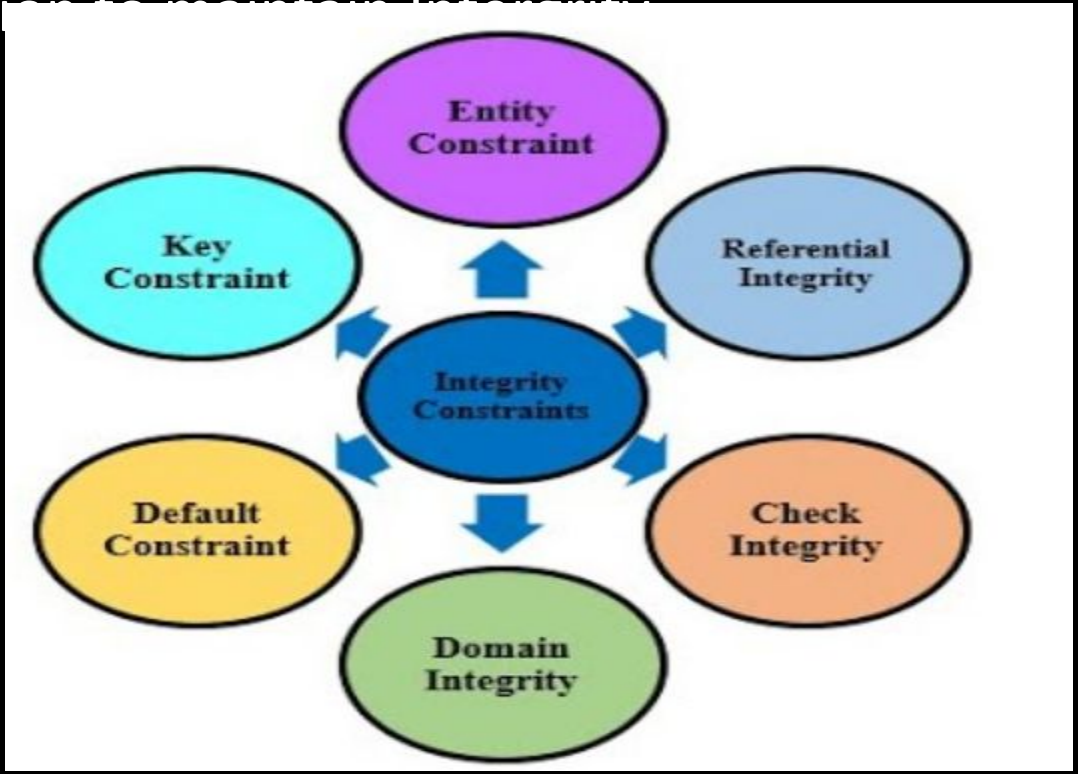
- The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.



Codd's 12 Rules

Rule 10: Integrity Independence

- Database should be able to apply integrity rules by using its query language.
- It should not be dependent on any external factor or applicati



Codd's 12 Rules

Rule 11: Distribution Independence

- A database should work properly regardless of its distribution across a network.
- The end-user should not be able to see that the data is distributed over many locations, they should always get the impression that the data is located at a single site only.

Rule 12: The Non-subversion Rule

- If a relational system allows low-level access, that law cannot be used to subvert or bypass the integrity rules to modify the data. This can be achieved with some sort of looking or encryption.

What are Anomalies?

- In Database Management Systems (DBMS),anomalies are data inconsistencies or irregularities that arise from poor database design, particularly due to data redundancy or poorly structured tables, leading to problems during data insertion, deletion, or update.

- Anomalies are deviations from the expected behavior or patterns in a database, often resulting in data inconsistencies.
- They can manifest as errors or inconsistencies that can lead to issues in data processing, analysis, and interpretation.
- Poorly designed databases, especially those with excessive data redundancy, are prone to anomalies.

Types of Anomalies:

- **Update Anomalies:** Occur when updating a piece of data requires changing it in multiple places, potentially leading to inconsistencies if some updates are missed.
- **Insertion Anomalies:** Occur when inserting new data requires also inserting redundant data, or when certain data cannot be inserted without also inserting data for other attributes.
- **Deletion Anomalies:** Occur when deleting a piece of data inadvertently deletes related data, leading to loss of information.

Causes of Anomalies:

- **Data Redundancy:** Storing the same data multiple times increases the risk of inconsistencies.
- **Poor Table Design:** Not properly normalizing the database can lead to anomalies.